

Konstrukturen

Begriff	Beschreibung
Definition	Ein Konstruktor ist eine spezielle Methode in einer Klasse, die beim Erstellen eines Objekts aufgerufen wird, um die Instanz der Klasse zu initialisieren.
Namensgebung	Der Konstruktor muss denselben Namen wie die Klasse haben. Er hat keinen Rückgabewert, nicht einmal <code>void</code> .
Standardkonstruktor	Wenn kein Konstruktor explizit definiert wird, fügt der Compiler automatisch einen Standardkonstruktor ohne Parameter hinzu. Dieser setzt Instanzvariablen auf ihre Standardwerte (z. B. <code>0</code> für <code>int</code> , <code>null</code> für Referenztypen).
Parametrisierter Konstruktor	Ein Konstruktor, der Parameter entgegennimmt, um die Instanzvariablen mit benutzerdefinierten Werten zu initialisieren. Er ermöglicht mehr Flexibilität bei der Objektinitialisierung.
Überladung von Konstrukturen	Eine Klasse kann mehrere Konstrukturen mit unterschiedlichen Parameterlisten haben. Konstrukturen mit gleicher Signatur (selber Name) aber unterschiedlicher Anzahl oder Typ der Parameter.
Zugriffsmodifikatoren	Konstrukturen können mit Zugriffsmodifikatoren wie <code>public</code> , <code>protected</code> , <code>private</code> oder <code>default</code> versehen werden.
<code>public</code> Konstruktor	Der Konstruktor ist für alle Klassen sichtbar und kann in jeder Instanz von Objekten aufgerufen werden.
<code>private</code> Konstruktor	Der Konstruktor ist nur innerhalb der gleichen Klasse sichtbar. Wird oft bei Singleton-Designmustern verwendet, um die Instanziierung durch andere Klassen zu verhindern.
<code>protected</code> Konstruktor	Der Konstruktor ist nur innerhalb des gleichen Pakets oder von Unterklassen sichtbar.
<code>default</code> Konstruktor	Wenn kein Zugriffsmodifikator angegeben wird, ist der Konstruktor nur innerhalb des gleichen Pakets sichtbar.
Konstruktorverkettung (Konstruktoraufruf innerhalb der Klasse)	Mit <code>this(...)</code> kann ein Konstruktor einen anderen Konstruktor derselben Klasse aufrufen. Dies vermeidet redundante Initialisierungen und erleichtert die Wartung des Codes.
Konstruktorverkettung (außerhalb der Klasse)	Ein Konstruktor kann den Konstruktor der Superklasse mit <code>super(...)</code> aufrufen, um die Initialisierung der Oberklasse sicherzustellen.
<code>this</code> -Verwendung	Innerhalb eines Konstruktors kann <code>this</code> verwendet werden, um auf das aktuelle Objekt oder andere Konstrukturen innerhalb derselben Klasse zu verweisen.

Begriff	Beschreibung
<code>super</code> -Verwendung	Wird verwendet, um den Konstruktor der Superklasse aufzurufen. Dies ist erforderlich, wenn die Superklasse keinen Standardkonstruktor hat oder bestimmte Initialisierungen durchgeführt werden müssen.
<code>default</code> Konstruktor in abgeleiteten Klassen	Wenn eine Klasse von einer anderen erbt und der Konstruktor der Basisklasse nicht explizit aufgerufen wird, wird der Standardkonstruktor der Basisklasse aufgerufen, wenn dieser vorhanden ist.
Abstrakte Klassen und Konstrukturen	Abstrakte Klassen können Konstrukturen haben, aber diese können nicht direkt aufgerufen werden. Der Konstruktor der abstrakten Klasse muss über den Konstruktor einer konkreten Unterklasse aufgerufen werden.
Konstrukturen in Interfaces	Interfaces können keine Konstrukturen haben, da sie nicht instanziiert werden können. Abgeleitete Klassen, die das Interface implementieren, müssen ihren eigenen Konstruktor bereitstellen.
Fehlende Konstrukturen	Wenn eine Klasse keinen Konstruktor definiert, fügt der Compiler automatisch einen parameterlosen Standardkonstruktor hinzu, es sei denn, ein anderer Konstruktor ist vorhanden.
<code>final</code> Konstrukturen	Ein Konstruktor kann nicht als <code>final</code> deklariert werden, da er nicht überschrieben werden kann.
<code>static</code> Konstrukturen	Ein Konstruktor kann nicht als <code>static</code> deklariert werden, da Konstrukturen zum Erstellen von Instanzen der Klasse dienen und somit nicht zur Klasse selbst gehören.
<code>synchronized</code> Konstrukturen	Konstrukturen können mit <code>synchronized</code> versehen werden, um sicherzustellen, dass nur ein Thread gleichzeitig den Konstruktor ausführen kann. Dies ist jedoch selten erforderlich und sollte nur in Mehr-Threading-Umgebungen verwendet werden.
Instanziierung und Konstrukturen	Ein Konstruktor wird immer dann aufgerufen, wenn eine neue Instanz der Klasse erstellt wird. Das Aufrufen eines Konstruktors ist die einzige Möglichkeit, eine Instanz eines Objekts zu erzeugen.
Rekursive Konstrukturen	Konstrukturen können nicht rekursiv sein. Ein Konstruktor darf sich nicht selbst aufrufen, da dies zu einer unendlichen Rekursion führen würde.
Vererbung und Konstrukturen	Wenn eine Klasse von einer anderen erbt, wird der Konstruktor der Basisklasse automatisch aufgerufen, es sei denn, der Konstruktor der abgeleiteten Klasse ruft explizit einen anderen Konstruktor der Basisklasse auf.

Begriff	Beschreibung
Abstrakte Konstruktoren	Abstrakte Klassen können Konstruktoren haben, aber Instanzen der abstrakten Klasse können nicht direkt erstellt werden. Stattdessen muss der Konstruktor über einen Konstruktor der abgeleiteten konkreten Klasse aufgerufen werden.
Konstruktoren und Initialisierungsblöcke	Initialisierungsblöcke (<code>{ }</code>), die vor dem Aufruf des Konstruktors ausgeführt werden, können innerhalb der Klasse definiert werden, um gemeinsame Initialisierungen zu ermöglichen.
Konstruktoren mit Variablen Argumenten (Varargs)	Konstruktoren können Variablenlängen-Argumente (<code>varargs</code>) akzeptieren, die als Array behandelt werden. Dies ermöglicht, dass beliebig viele Parameter an den Konstruktor übergeben werden können.
Konstruktoren und Annotationen	Konstruktoren können mit Annotationen wie <code>@Deprecated</code> oder <code>@Override</code> versehen werden, um zusätzliche Metadaten zu deklarieren, wie etwa, dass der Konstruktor veraltet ist oder überschrieben wird.
Exception Handling im Konstruktor	Konstruktoren können mit <code>try-catch</code> -Blöcken versehen werden, um Fehler zu behandeln, die während der Initialisierung auftreten können. Falls der Konstruktor eine Ausnahme wirft, wird die Instanziierung des Objekts abgebrochen.

Zusammenfassung

- **Konstruktoren** sind entscheidend für die Instanziierung und Initialisierung von Objekten in Java.
- Sie **können überladen** werden, was mehrere Konstruktoren in einer Klasse mit unterschiedlichen Parameterlisten ermöglicht.
- Der Konstruktor einer Klasse **ruft oft den Konstruktor der Superklasse auf** (mit `super()`), wenn die Klasse vererbt wird.
- Konstruktoren können auch mit **Modifikatoren** wie `public`, `private`, `protected` versehen werden, um den Zugriff zu steuern.
- In **abstrakten Klassen** können Konstruktoren existieren, aber sie können nicht direkt instanziiert werden.
- **Konstruktoren in vererbten Klassen** müssen explizit oder implizit den Konstruktor der Basisklasse aufrufen, wenn erforderlich.